

Implémentation, ServeurMultiThreade (v5)

Fabrice.Kordon@lip6.fr



Objectif de cette version

2

On casse tout

- Des pools partout
- Des groupes de threads partout
- Associer les pools à des groupes de thread
 - ▶ Le serveur
 - ▶ Les clients
 - ▶ Les servants
- Gestion revisitée de la terminaison
 - ▶ S'appuyer sur les pools
- Meilleure gestion de la terminaison
 - ▶ Bien plus élégante



Utilitaires et ce qui ne change pas

3

```
import java.util.Random;  
import java.util.concurrent.ArrayBlockingQueue;  
import java.util.concurrent.ExecutorService;  
import java.util.concurrent.Executors;  
import java.util.concurrent.ThreadFactory; // nouvelle dépendance  
import java.lang.ThreadGroup; // nouvelle dépendance  
import java.util.concurrent.TimeUnit; // nouvelle dépendance
```

```
class Requete {...}
```

```
class Servant implements Runnable {...}
```

La classe monUsineDeThread

4

Double objectif

-  Créer une thread
-  L'associer à un groupe donné

```
class MonUsineDeThread implements ThreadFactory {  
  
    private ThreadGroup monGroupe;  
  
    MonUsineDeThread (ThreadGroup g) {  
        monGroupe = g;  
    }  
  
    public Thread newThread (Runnable r) {  
        return new Thread(monGroupe, r);  
    }  
}
```


La classe LeServeur

5

```
class LeServeur implements Runnable {
    private ArrayBlockingQueue<Requete> f;
    // Borner le parallélisme des servants à 3
    private final int MAX_SERV = 3;
    private ExecutorService executor;
    ThreadGroup groupeServants = new ThreadGroup("servants");

    private ExecutorService poolClients; // les pools client

    LeServeur (ArrayBlockingQueue<Requete> fr, ExecutorService pc) {
        f = fr;
        MonUsineDeThread monUsine = new MonUsineDeThread(groupeServants);
        executor = Executors.newFixedThreadPool(MAX_SERV, monUsine);
        poolClients = pc;
    }

    private void trace (String m) {...}
}
```


La classe LeServeur

5

```
public void run() {
    Requete r = null;
    this.trace("initialisé");
    while (!poolClients.isTerminated()) {
        try {
            r = f.poll(1, TimeUnit.SECONDS); // Pour terminaison
            if (r != null) {
                this.trace("je traite " + r);
                Servant s = new Servant(r);
                executor.execute(s);
            } else {
                this.trace("je m'endors, pool terminé = " + poolClients.isTerminated());
                Thread.sleep(500);
            }
        } catch (InterruptedException e) {
            System.err.println ("Serveur, ne doit jamais arriver, " + e);
            return;
        }
    }
    groupeServants.list();
    executor.shutdown();
    this.trace("Adios");
}
```


La classe UnClient

6

```
class UnClient implements Runnable {
    private int monId;
    private ArrayBlockingQueue<Requete> f;

    private Random generateur = new Random();
    private final Object reveil = new Object(); // attente réponse

    private static int cpt = 1;
    private static final Object mutex = new Object();

    UnClient (ArrayBlockingQueue<Requete> fr) {
        f = fr;
        synchronized (mutex) {
            monId = cpt++;
        }
    }

    private void trace (String m) {...}

    public void requeteTraitee () {...}

    public String toString () {...}
}
```


La classe UnClient

6

```
public void run() {  
    this.trace("initialisé");  
    Requete r = new Requete(this, generateur.nextInt(1000));  
    this.trace("j'envoie la requête " + r);  
    try {  
        f.put (r);  
        this.trace("j'attends le serveur");  
        synchronized (veille) {  
            veille.wait();  
        }  
    } catch (InterruptedException e) {  
        System.out.println ("UnClient "+monId+", cela doit jamais arriver, " + e);  
    } // plus de terminaison du serveur  
    this.trace("ma requête a été traitée");  
}
```


La «classe principale»

7

```
public class ServeurMultiThreade5 {
    static int tailleFile;
    static int nbClients;

    public static void main(String []args) {
        try {
            tailleFile = Integer.parseInt(args[0]);
            nbClients = Integer.parseInt(args[1]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments requis, taille de la file, nombre de clients.");
            return;
        }
        // Créer la file de requête
        ArrayBlockingQueue<Requete> laFile =
            new ArrayBlockingQueue<Requete>(tailleFile, true);

        // Créer les groupes du serveur et des clients
        ThreadGroup groupeServeur = new ThreadGroup("serveur");
        MonUsineDeThread usineServeur = new MonUsineDeThread(groupeServeur);
        ExecutorService execServeur = Executors.newSingleThreadExecutor(usineServeur);

        ThreadGroup groupeClients = new ThreadGroup("clients");
        MonUsineDeThread usineClients = new MonUsineDeThread(groupeClients);
        ExecutorService execClients = Executors.newFixedThreadPool(5, usineClients);
    }
}
```


La «classe principale»

7

```
// Lancer le serveur dans son groupe
execServeur.execute(new LeServeur(laFile, execClients));
// Lancer les clients
for (int i = 0; i < nbClients ; i++) {
    execClients.execute(new UnClient(laFile));
}
// Terminer le système (quand tout aura été exécuté)
groupeServeur.list();
groupeClients.list();
execClients.shutdown();
execServeur.shutdown();
```

```
}
```

```
}
```


Première exécution

8

```
$ java ServeurMultiThreade5 2 4
Serveur : initialisé
UnClient 1 : initialisé
UnClient 2 : initialisé
UnClient 3 : initialisé
java.lang.ThreadGroup[name=serveur,maxpri=10]
  UnClient 4 : initialisé
Thread[Thread-0,5,serveur]
java.lang.ThreadGroup[name=clients,maxpri=10]
  Thread[Thread-1,5,clients]
  Thread[Thread-2,5,clients]
UnClient 2 : j'envoie la requête (client 2, 78)
UnClient 4 : j'envoie la requête (client 4, 97)
UnClient 3 : j'envoie la requête (client 3, 761)
UnClient 1 : j'envoie la requête (client 1, 541)
UnClient 2 : j'attends le serveur
  Thread[Thread-3,5,clients]
Serveur : je traite (client 2, 78)
UnClient 3 : j'attends le serveur
UnClient 4 : j'attends le serveur
  Thread[Thread-4,5,clients]
Serveur : je traite (client 4, 97)
UnClient 1 : j'attends le serveur
Servant (client 2, 78) : initialisé
Serveur : je traite (client 3, 761)
Servant (client 4, 97) : initialisé
Serveur : je traite (client 1, 541)
Servant (client 3, 761) : initialisé
Servant (client 2, 78) : je préviens le client
Servant (client 1, 541) : initialisé
UnClient 2 : ma requête a été traitée
Servant (client 4, 97) : je préviens le client
```

```
UnClient 4 : ma requête a été traitée
Servant (client 1, 541) : je préviens le client
UnClient 1 : ma requête a été traitée
Servant (client 3, 761) : je préviens le client
UnClient 3 : ma requête a été traitée
Serveur : je m'endors, pool terminé = true
java.lang.ThreadGroup[name=servants,maxpri=10]
  Thread[Thread-5,5,servants]
  Thread[Thread-6,5,servants]
  Thread[Thread-7,5,servants]
Serveur : Adios
```



Observation

La taille maximum du pool de client n'est pas atteinte

Seconde exécution



Observation

À l'affichage du groupe des clients, le pool est «saturé»

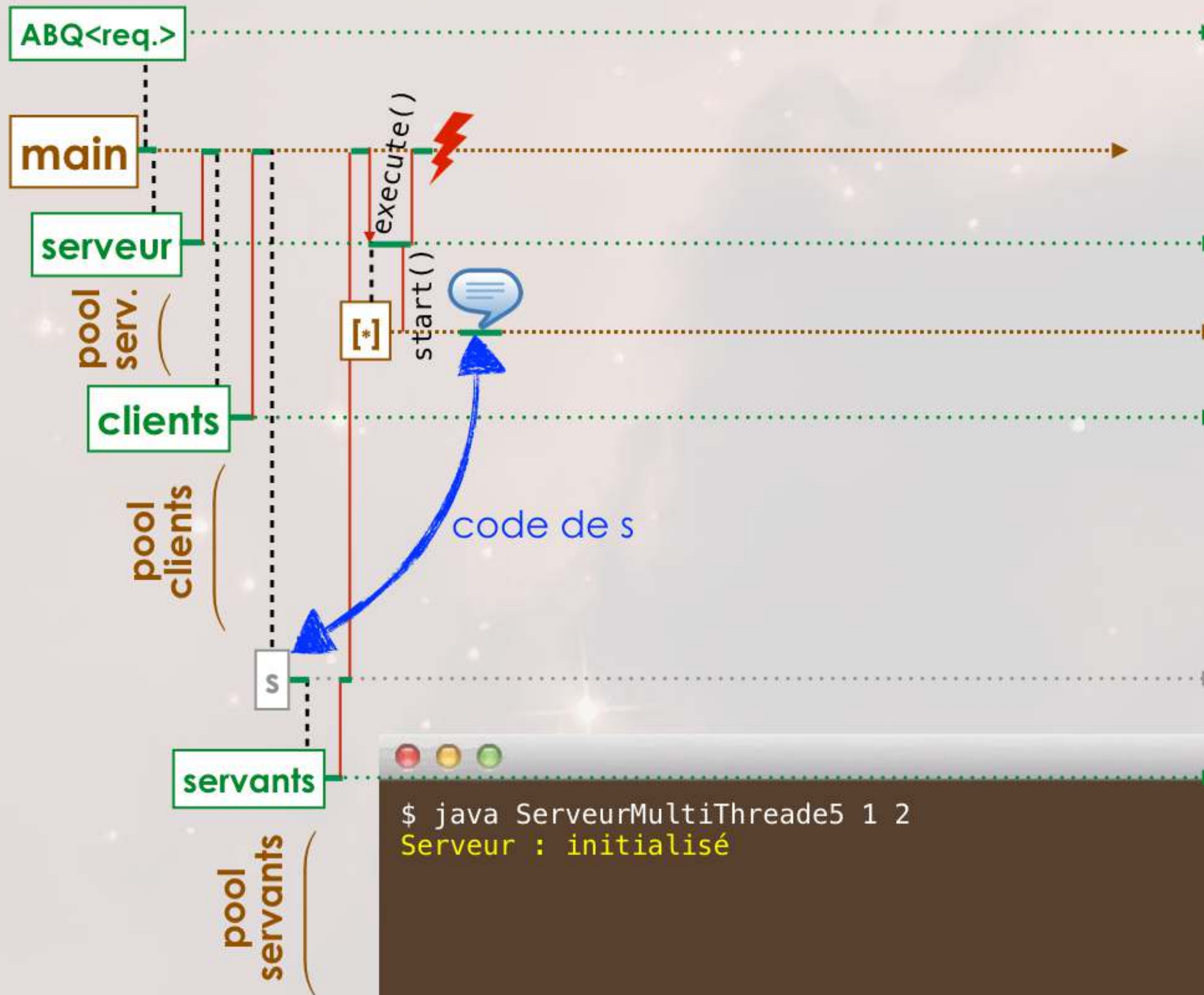
```
UnClient 4 : initialisé
java.lang.ThreadGroup[name=serveur,maxpri=10]
UnClient 5 : initialisé
UnClient 4 : j'envoie la requête (client 4, 182)
UnClient 1 : j'envoie la requête (client 1, 645)
UnClient 5 : j'envoie la requête (client 5, 313)
UnClient 2 : j'envoie la requête (client 2, 535)
UnClient 3 : j'envoie la requête (client 3, 73)
Serveur : je traite (client 4, 182)
UnClient 5 : j'attends le serveur
UnClient 4 : j'attends le serveur
UnClient 1 : j'attends le serveur
Thread[Thread-0,5,serveur]
java.lang.ThreadGroup[name=clients,maxpri=10]
Thread[Thread-1,5,clients]
Thread[Thread-2,5,clients]
Thread[Thread-3,5,clients]
Thread[Thread-4,5,clients]
Thread[Thread-5,5,clients]
Serveur : je traite (client 1, 645)
UnClient 2 : j'attends le serveur
Servant (client 4, 182) : initialisé
Serveur : je traite (client 5, 313)
UnClient 3 : j'attends le serveur
Servant (client 1, 645) : initialisé
Serveur : je traite (client 2, 535)
Servant (client 5, 313) : initialisé
```

```
Serveur : je traite (client 3, 73)
Servant (client 4, 182) : je prévient le client
UnClient 4 : ma requête a été traitée
Servant (client 2, 535) : initialisé
UnClient 6 : initialisé
UnClient 6 : j'envoie la requête (client 6, 105)
UnClient 6 : j'attends le serveur
Serveur : je traite (client 6, 105)
Servant (client 5, 313) : je prévient le client
UnClient 5 : ma requête a été traitée
Servant (client 3, 73) : initialisé
UnClient 7 : initialisé
UnClient 7 : j'envoie la requête (client 7, 27)
UnClient 7 : j'attends le serveur
Serveur : je traite (client 7, 27)
Servant (client 3, 73) : je prévient le client
UnClient 3 : ma requête a été traitée
Servant (client 6, 105) : initialisé
Servant (client 6, 105) : je prévient le client
Servant (client 7, 27) : initialisé
UnClient 6 : ma requête a été traitée
Servant (client 7, 27) : je prévient le client
UnClient 7 : ma requête a été traitée
Servant (client 1, 645) : je prévient le client
UnClient 1 : ma requête a été traitée
Servant (client 2, 535) : je prévient le client
UnClient 2 : ma requête a été traitée
Serveur : je m'endors, pool terminé = true
java.lang.ThreadGroup[name=servants,maxpri=10]
Thread[Thread-6,5,servants]
Thread[Thread-7,5,servants]
Thread[Thread-8,5,servants]
Serveur : Adios
```




Un petit gros chronogramme pour la route?

10

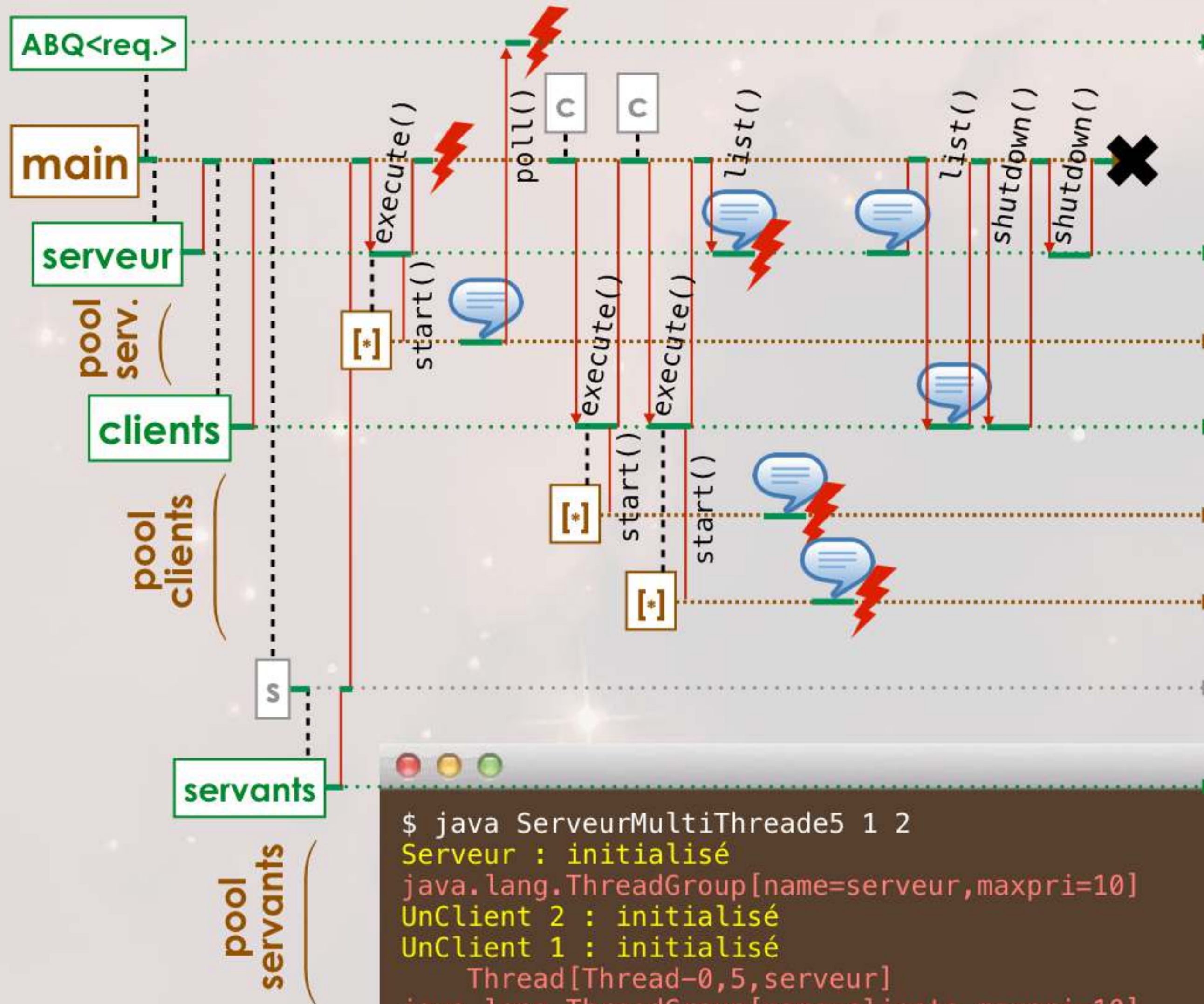


```
$ java ServeurMultiThreade5 1 2
Serveur : initialisé
```




Un petit gros chronogramme pour la route?

10

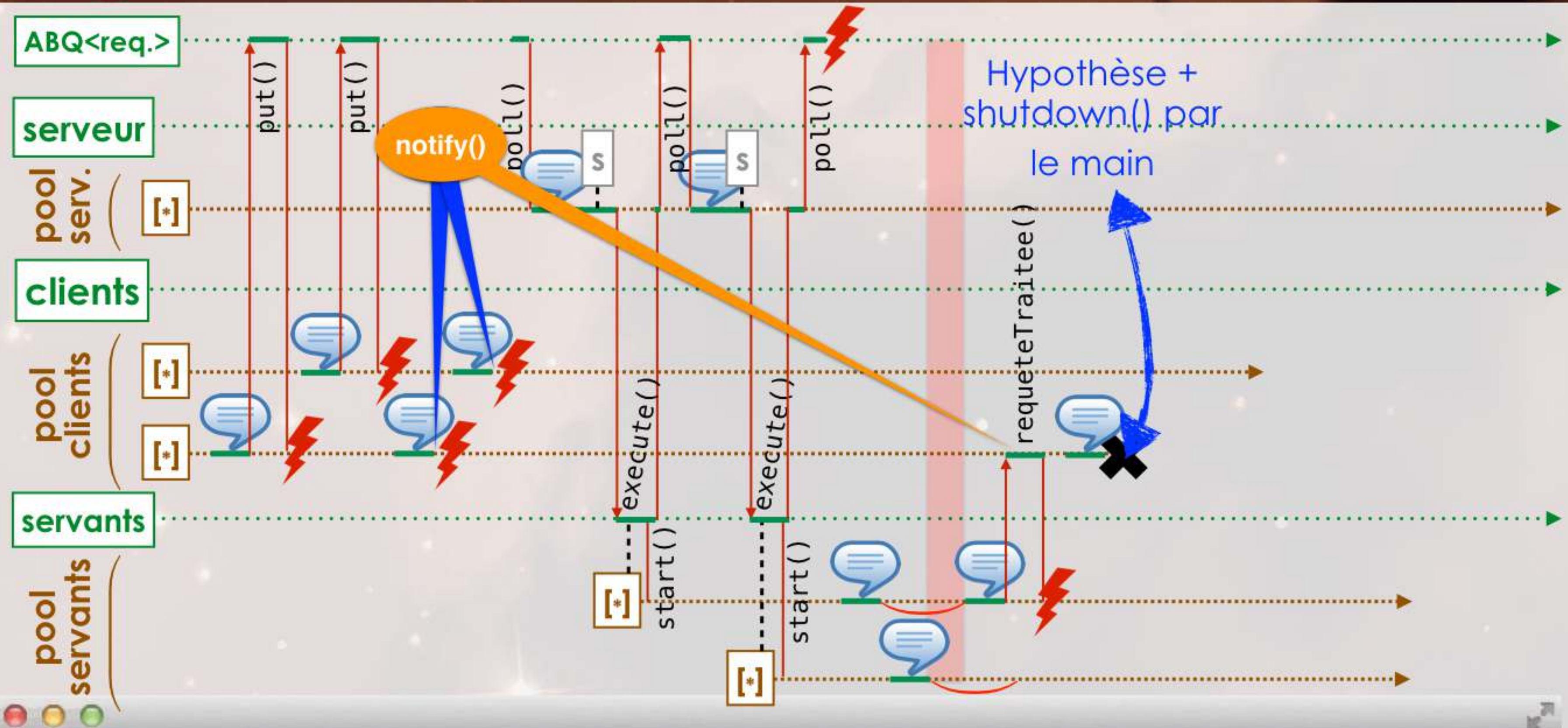


```
$ java ServeurMultiThreade5 1 2
Serveur : initialisé
java.lang.ThreadGroup[name=serveur,maxpri=10]
UnClient 2 : initialisé
UnClient 1 : initialisé
  Thread[Thread-0,5,serveur]
java.lang.ThreadGroup[name=clients,maxpri=10]
  Thread[Thread-1,5,clients]
  Thread[Thread-2,5,clients]
```




Suite du petit gros chronogramme

11

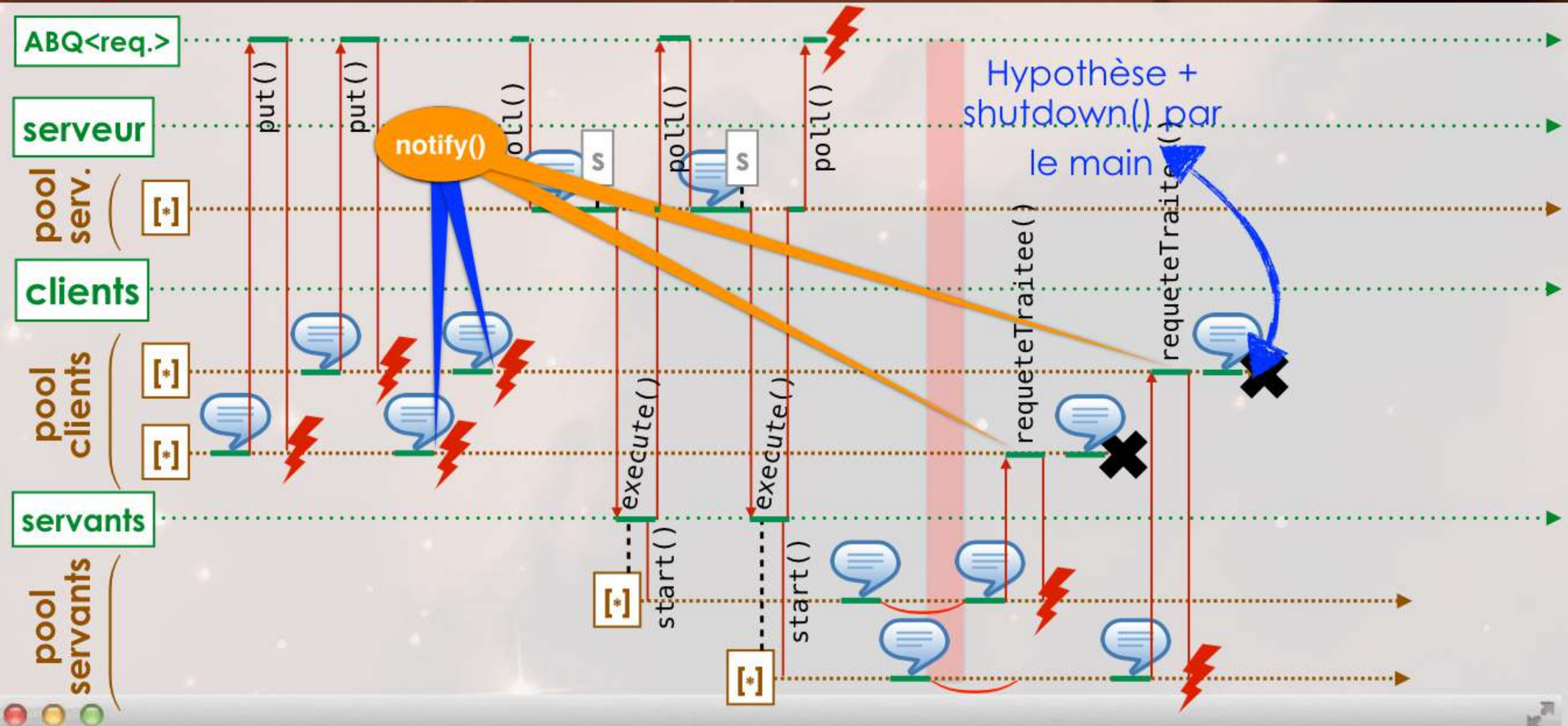


```
UnClient 1 : j'envoie la requête (client 1, 40)
UnClient 2 : j'envoie la requête (client 2, 823)
UnClient 1 : j'attends le serveur
UnClient 2 : j'attends le serveur
Serveur : je traite (client 1, 40)
Serveur : je traite (client 2, 823)
Servant (client 1, 40) : initialisé
Servant (client 2, 823) : initialisé
Servant (client 1, 40) : je prévien le client
UnClient 1 : ma requête a été traitée
```




Suite du petit gros chronogramme

11



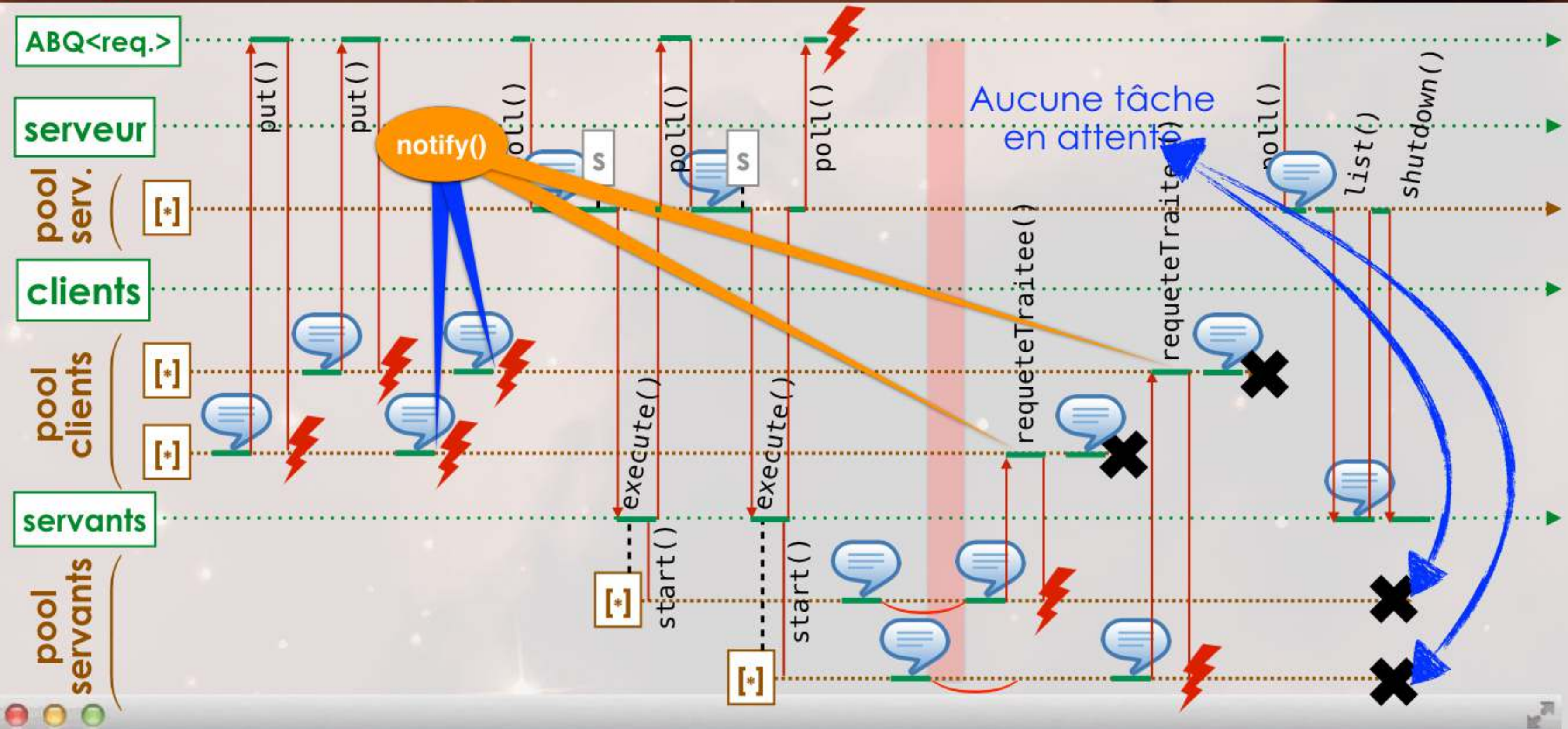
```
UnClient 1 : j'envoie la requête (client 1, 40)
UnClient 2 : j'envoie la requête (client 2, 823)
UnClient 1 : j'attends le serveur
UnClient 2 : j'attends le serveur
Serveur : je traite (client 1, 40)
Serveur : je traite (client 2, 823)
Servant (client 1, 40) : initialisé
Servant (client 2, 823) : initialisé
Servant (client 1, 40) : je préviens le client
UnClient 1 : ma requête a été traitée
```

```
Servant (client 2, 823) : je préviens le client
UnClient 2 : ma requête a été traitée
```




Suite du petit gros chronogramme

11



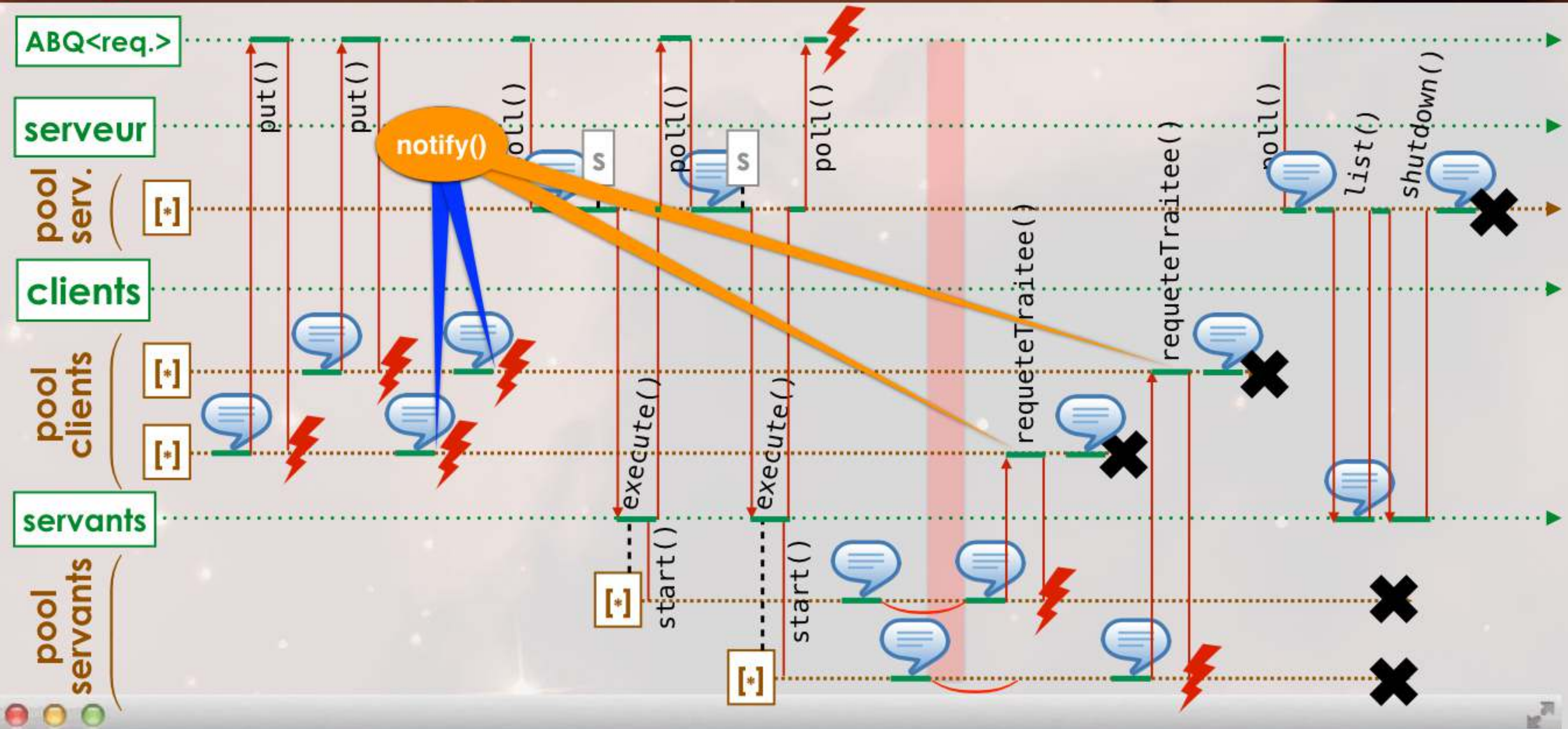
```
UnClient 1 : j'envoie la requête (client 1, 40)
UnClient 2 : j'envoie la requête (client 2, 823)
UnClient 1 : j'attends le serveur
UnClient 2 : j'attends le serveur
Serveur : je traite (client 1, 40)
Serveur : je traite (client 2, 823)
Servant (client 1, 40) : initialisé
Servant (client 2, 823) : initialisé
Servant (client 1, 40) : je prévien le client
UnClient 1 : ma requête a été traitée
```

```
Servant (client 2, 823) : je prévien le client
UnClient 2 : ma requête a été traitée
Serveur : je m'endors, pool terminé = true
java.lang.ThreadGroup[name=servants,maxpri=10]
  Thread[Thread-3,5,servants]
  Thread[Thread-4,5,servants]
Serveur : Adios
```




Suite du petit gros chronogramme

11



```
UnClient 1 : j'envoie la requête (client 1, 40)
UnClient 2 : j'envoie la requête (client 2, 823)
UnClient 1 : j'attends le serveur
UnClient 2 : j'attends le serveur
Serveur : je traite (client 1, 40)
Serveur : je traite (client 2, 823)
Servant (client 1, 40) : initialisé
Servant (client 2, 823) : initialisé
Servant (client 1, 40) : je prévien le client
UnClient 1 : ma requête a été traitée
```

```
Servant (client 2, 823) : je prévien le client
UnClient 2 : ma requête a été traitée
Serveur : je m'endors, pool terminé = true
java.lang.ThreadGroup[name=servants,maxpri=10]
  Thread[Thread-3,5,servants]
  Thread[Thread-4,5,servants]
Serveur : Adios
```


En guise de conclusion...

12

Génial non?

- Prise en charge de la terminaison par les groupes de threads
- Observation et organisation des threads via les groupes

